Agent Reliability in Financial Tool Use: A Layered Orchestration Approach

Abstract

AI agents completing financial operations using tool-calling interfaces fail at a 75% rate out of the box. The failures span two categories: execution errors (malformed arguments and broken data chaining between steps) and tool selection errors (the agent calls the wrong tool entirely). Both failure types are non-deterministic — the same model can pass or fail across runs — and both are model-independent, appearing in Claude Sonnet 4 and GPT-4o alike.

This paper presents a layered orchestration architecture that brings the failure rate to zero across multiple model providers through five complementary mechanisms: argument validation, structured retry, schema-rich tool descriptions, typed output labeling, and tool selection correction. Each layer addresses a distinct failure category. No single layer is sufficient alone.

1. The Evaluation Gap

Existing LLM evaluation frameworks (Braintrust, Langsmith, Promptfoo, DeepEval) test prompt-level safety: can the model reject injection attacks, produce accurate outputs, follow instructions? These are necessary but miss the dominant failure mode in financial tool use: the agent calls the right tool with the wrong arguments.

Agent protocol frameworks (ACK, MPP, ERC-8004) define the tools and message formats but provide no evaluation infrastructure. They ship demos and unit tests.

None of them test whether an AI agent can autonomously complete a multi-step workflow using their tools, measure the failure rate, or provide an orchestration layer to improve it. To my knowledge, this is the first framework that evaluates agent reliability at the workflow level using real financial cryptographic operations, identifies the specific failure taxonomy, validates the findings across multiple model providers, and provides a layered solution that achieves 100% completion.

Financial workflows are multi-step and cryptographically chained. A credential signing step produces a JWT that must be passed intact to a verification step. A payment request must include exact field specifications (amount, decimals,

currency, recipient) or the tool rejects it. These are not reasoning failures. They are execution failures, and they require a different evaluation and mitigation approach.

2. Failure Taxonomy

Testing Claude Sonnet 4 and GPT-4o against four financial workflows (identity credential lifecycle, payment request lifecycle, untrusted issuer rejection, wrong issuer rejection) revealed three failure categories. The first two appeared in both models.

Category 1: Argument Construction. The agent omits required fields or passes wrong types. Example: creating a payment request without the `decimals` field, which is required by the payment schema.

The agent understands the workflow but cannot reliably populate all required fields from memory.

Category 2: Argument Chaining. The agent constructs valid arguments for each tool individually but fails to correctly pass output from one step as input to the next. Example: a signing step produces a 600-character JWT string, but the verification step receives a truncated or restructured version.

Cryptographic values are binary: one wrong character and verification fails.

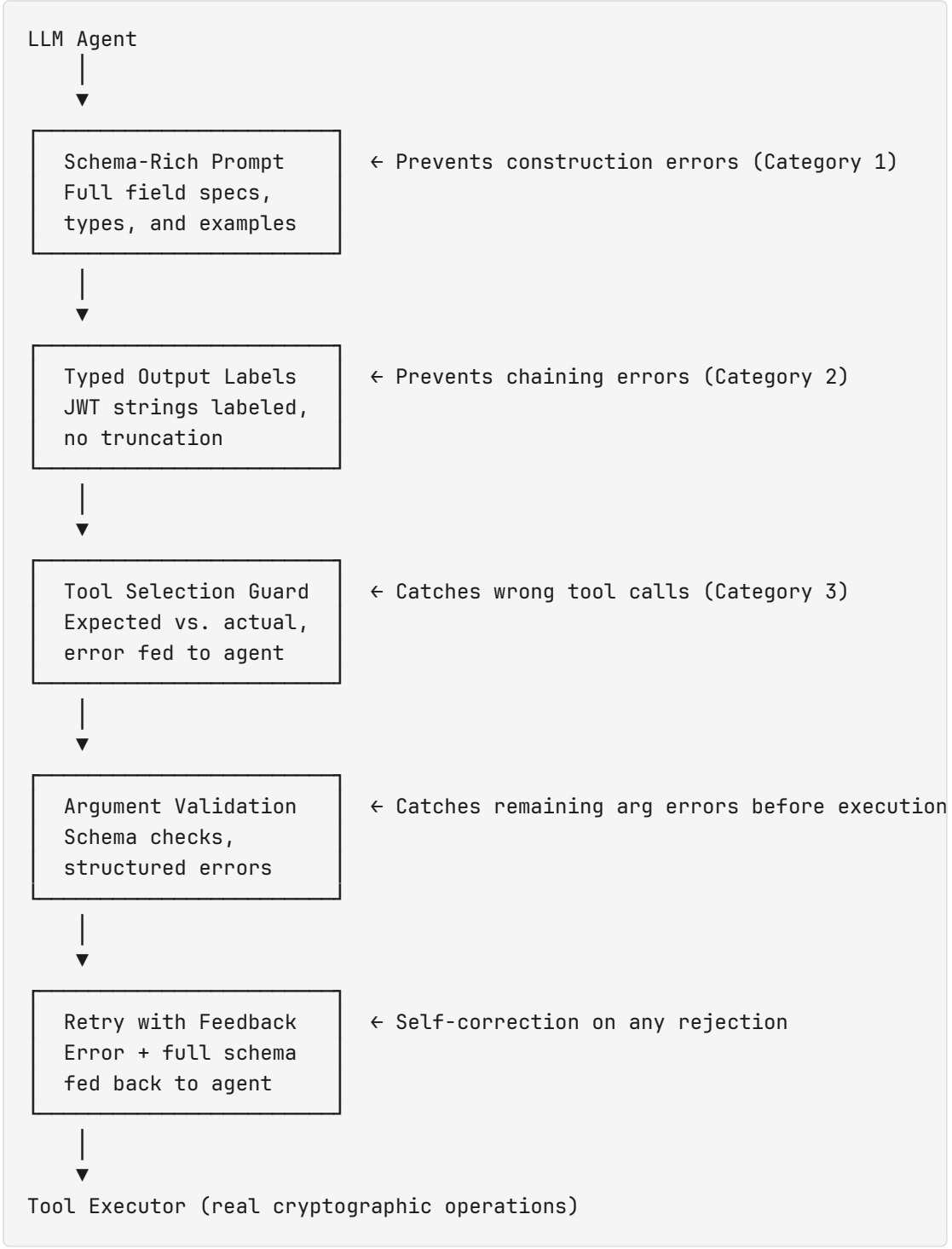
Category 3: Tool Selection. The agent selects the wrong tool for a given step. Example: when asked to create a controller credential, the agent calls `ack_generate_keypair` instead of `ack_create_controller_credential`. This failure is non-deterministic — Claude Sonnet 4 passes the same test on some runs and fails on others. GPT-4o did not exhibit this failure in observed runs, but the non-deterministic nature means it cannot be ruled out.

This category was initially assumed never to occur. Multi-run testing revealed it as an intermittent failure that single-run evaluations miss entirely. Most evaluation frameworks only test this category, but only once per eval — missing the non-determinism.

Categories 1, 2, and 3 account for all observed failures across both providers.

3. The Layered Architecture

Five mechanisms, each addressing a different failure category:



3.1 Schema-Rich Prompting

Replace terse tool descriptions with complete field specifications including types, required markers, and examples. Instead of:

```
ack_create_payment_request: Args: { paymentOptions: [...], jwk, did }
```

Provide:

```
ack_create_payment_request
  Required: {
    "paymentOptions": [{
      "id": "option-1",
      "amount": 1.00,
      "decimals": 6,
      "currency": "USDC",
      "recipient": "0x1234...5678"
    }],
    "jwk": "<JWK string from keypair>",
    "did": "<requester DID>"
  }
  Note: every payment option MUST include id, amount, decimals,
  currency, recipient.
```

Impact: Eliminated all argument construction errors. Zero validation failures after this change.

3.2 Typed Output Labels

When presenting previous step results to the agent, label outputs by their data type rather than dumping raw text:

- JWT strings: `"JWT string (pass this entire value as-is): eyJ..."`
- JSON objects: passed through as-is
- Errors: `"ERROR: ..."`

Critically, do not truncate cryptographic values. JWTs are 400-800 characters.

Truncation produces cryptographically invalid input that will fail downstream verification. The agent correctly passes full JWT strings between signing and verification steps after this change.

3.3 Tool Selection Guard

Before executing a tool call, verify the agent selected the expected tool for the current workflow step. When the agent calls the wrong tool, return a structured error identifying the correct tool and allow a retry:

```
Wrong tool selected. You called "ack_generate_keypair" but this step
requires "ack_create_controller_credential". Please call
"ack_create_controller_credential" with the correct arguments.
```

This layer addresses Category 3 failures, which are non-deterministic and invisible to single-run evaluations. The same model can select the correct tool on one run and the wrong tool on the next. Without this layer, intermittent tool selection errors cause unpredictable workflow failures in production.

Impact: Claude Sonnet 4 intermittently fails `wf-fail-001` by calling `ack_generate_keypair` instead of `ack_create_controller_credential`. With tool selection correction and retry, the agent self-corrects to 100% on every run.

3.4 Argument Validation

Check tool arguments against schemas before execution. When arguments are invalid, return a structured error message that includes the specific missing fields and the full required schema:

```
Validation failed. Fix ALL of these issues and retry:
- paymentOptions[0].decimals: Each payment option must have
  decimals (e.g. 6 for USDC)

Full required schema for ack_create_payment_request:
{
  "paymentOptions": [{ "id": string, "amount": any,
    "decimals": number, "currency": string,
    "recipient": string }],
  "jwk": string,
  "did": string
}
```

The validation layer also produces an audit log with timestamps, tool names, and field-level errors for every rejected call. The audit log provides compliance data showing exactly when and how the agent attempted malformed operations.

3.5 Retry with Feedback

When validation rejects a tool call, feed the error (including the failed attempt) back to the agent as context and allow additional attempts. The agent sees what it did wrong and the full schema of what's expected.

Impact: Allows self-correction. In testing, retry improved pass rate from 25% to 50% before the other layers eliminated the need for it.

4. Results

4.1 Layer Progression (Claude Sonnet 4)

Configuration	Pass Rate	Validation Errors
Baseline (no layers)	25%	N/A
+ Validation	25%	2
+ Validation + Retry	50%	4 (across retries)
+ Improved Prompt	50%	0
+ Output Labeling	**100%**	**0**

4.2 Multi-Model Validation

The framework was validated across two model providers to confirm the findings are architectural, not model-specific.

Prompt-Level Evals (security, accuracy, adversarial):

Provider	Pass Rate	Critical Failures
Baseline (naive filter)	69.2% (9/13)	2
Claude Sonnet 4	84.6% (11/13)	1
GPT-4o	84.6% (11/13)	1

Both models score identically and fail on the same two tests — an over-rejection of base64-encoded memo content and over-validation of address format. The failure patterns are identical across providers.

Workflow Evals (raw, without orchestration):

Provider	Pass Rate	Notes
Claude Sonnet 4	75–100%	Non-deterministic. Intermittently selects wrong tool on `wf-fail-001`.
GPT-4o	100%	Consistent across observed runs.

Workflow Evals (with orchestration layers):

Provider	Pass Rate
Claude Sonnet 4 + Orchestration	**100%** (4/4)
GPT-4o + Orchestration	**100%** (4/4)

All five layers together achieve 100% pass rate across both providers on a suite of four financial workflows covering identity credentials, payment requests, failure recognition, and issuer verification.

4.3 Non-Determinism

A critical finding: raw LLM results are non-deterministic across runs. Claude Sonnet 4 passes `wf-fail-001` on some runs and fails on others by selecting the wrong tool. This means single-run evaluations produce unreliable results. The orchestration layer eliminates this variance by catching and correcting tool selection errors through retry with feedback. This is a stronger guarantee than "the model usually gets it right."

5. Production Application

Deployment Architecture

The orchestration layer serves as both the evaluation tool and the production reliability layer. The same code that evaluates agent reliability during development enforces it in production:

- Tool selection validation catches wrong tool calls before execution
- Argument schema validation catches malformed arguments before execution

- Structured retry with error feedback allows self-correction without human intervention
- The audit log captures every validation failure and self-correction for compliance
- The deterministic baseline (runs in under 1 second with no API calls) serves as a regression gate in CI

Evaluation Pipeline

1. **Pre-deployment:** Run deterministic baseline (must pass, proves tools work). Run LLM eval across providers (sets reliability expectation).
2. **Post-update:** Re-run on model or tool changes. Regression is a blocking issue. Multi-run testing required to catch non-deterministic failures.
3. **Continuous:** Schedule evals against production endpoints. Track validation error rates and self-correction frequency for drift detection.
4. **Model comparison:** The framework is model-agnostic. The same eval suite runs against Claude, GPT-4o, and any provider that supports chat completion. Provider-specific failure patterns (e.g., Claude's intermittent tool selection errors) are surfaced automatically.

6. Closing Thoughts

As far as I can tell, no existing framework evaluates agent reliability at the workflow level using real financial cryptographic operations. The failure taxonomy here (argument construction, argument chaining, and tool selection) came from observing actual failures across multiple model providers, not from theory. Most eval frameworks only test reasoning — a category that accounts for none of the observed failures.

The eval tools and the production orchestration layer are the same code. The validation, retry, tool selection correction, and output labeling that fix eval failures prevent production failures. The progression from 25% to 100% is reproducible, each layer's contribution is independently measurable, and the results hold across both Claude Sonnet 4 and GPT-4o.

A key finding is that single-run evaluations are insufficient for financial workflows. Non-deterministic tool selection errors mean a model can pass 100% on one run and fail on the next. Multi-run evaluation with orchestration-layer correction is necessary for any reliability guarantee.

There's plenty left to figure out. Multi-agent workflows, where two parties transact with their own credentials and verify each other, are the obvious next step. Adversarial tool use is an open question: can prompt injection via workflow context trick an agent into redirecting a payment or skipping verification? The current suite tests 3-4 step workflows. Real financial operations involve 8-12 steps with branching logic. Latency budgets and cost-per-completion metrics are needed before any of this is production-ready.